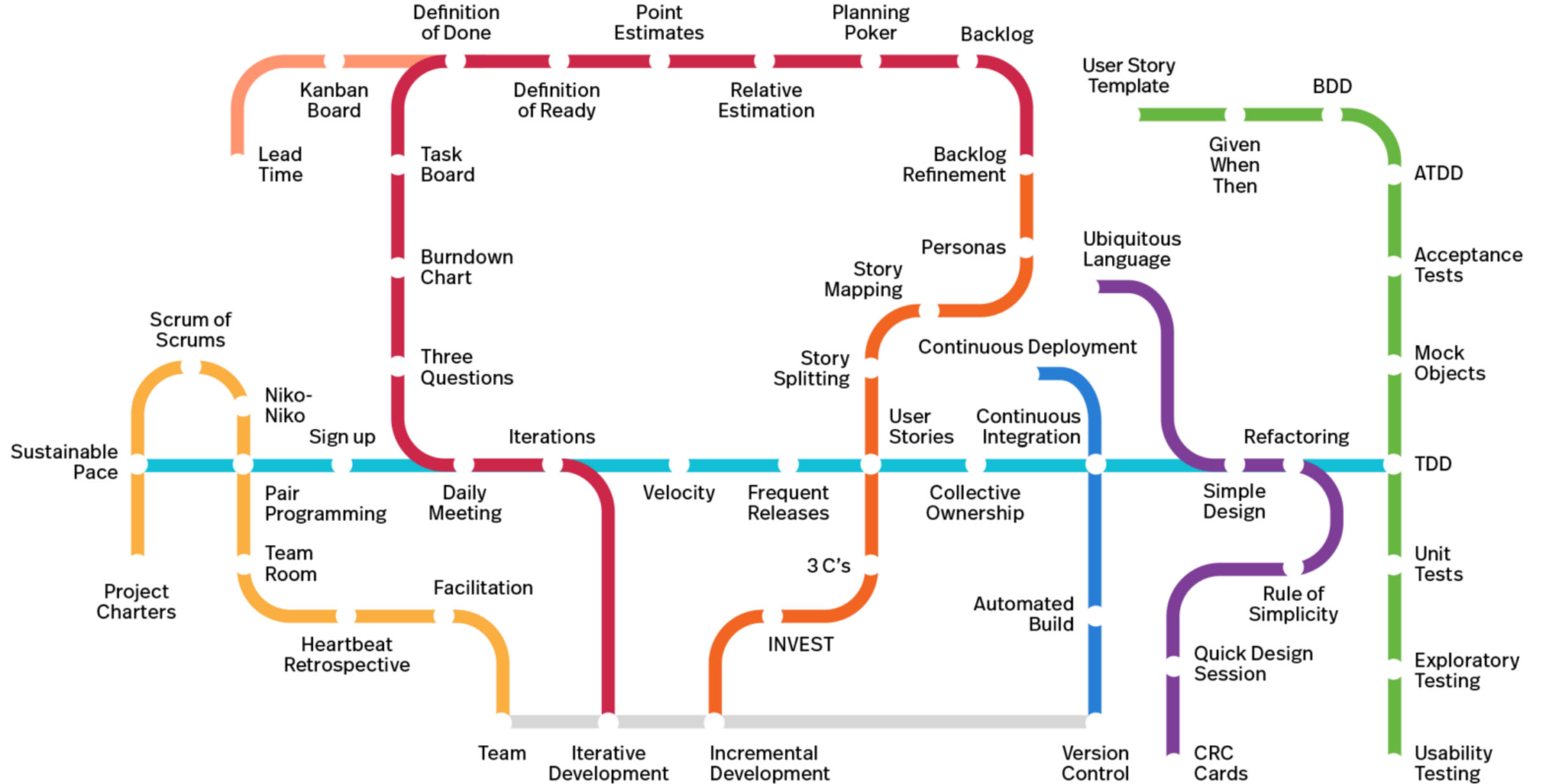




The Agile Subway Map

Lecturer: Adel Vahdati



Extreme Programming

Teams

Lean

Scrum

Product Management

DevOps

Design

Testing

Fundamentals



Why Scrum Alone Is Not Enough

- Does Scrum tell us how to write good code?
- **Scrum defines:**
 - Roles
 - Events
 - Artifacts
- **Scrum does not define:**
 - Engineering practices
 - Technical quality
 - Deployment & operations
 - Organizational change



Common Misconception

- “If we do Scrum, we are Agile.”
- **Reality:**
 - Scrum is a framework
 - Agile is a mindset + collection of practices
 - Teams must choose and adapt practices



Cargo-Cult Scrum

- **When Scrum rituals exist, but agility doesn't**
- **What it is**
 - Copying Scrum practices without understanding their purpose
 - Focus on ceremonies instead of outcomes
- **What it looks like**
 - Stand-ups = status reports
 - Retrospectives with no change
 - Roles without real authority
 - Backlogs filled to "run a sprint"



Cargo-Cult Scrum

- **Why it's dangerous**

- Creates a false sense of agility
- Hides real problems
- Lowers team motivation
- No improvement in value delivery

- **How to avoid it**

- Focus on why, not just how
- Apply Transparency, Inspection, Adaptation
- Empower teams
- Measure success by customer value

- **Key message:** If Scrum is just rituals, it's not Scrum — it's Cargo-Cult Scrum.

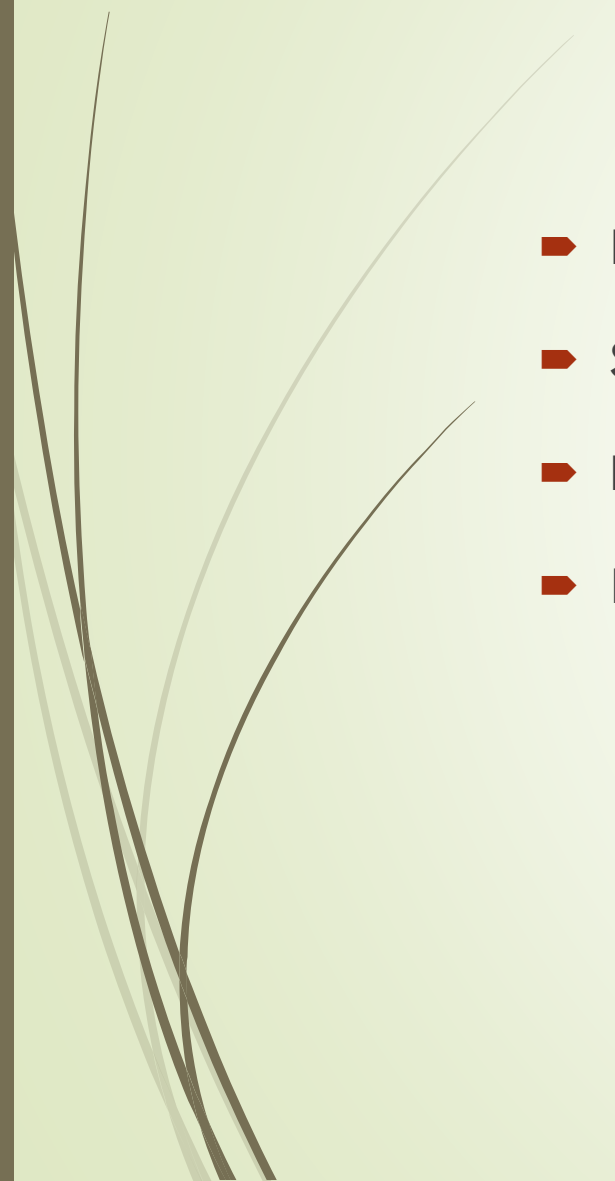


What Is the Agile Subway Map

- A visual metaphor for Agile practices
 - Created to show:
 - Diversity of Agile approaches
 - Relationships between practices
 - No single “correct” path
- 



The Subway Map Metaphor

- **Lines** → Agile approaches / philosophies
 - **Stations** → Practices or techniques
 - **Intersections** → Shared practices
 - **Routes** → Team-specific Agile journey
- 



Why a “Subway Map” Works Well

- **Agile adoption is:**

- **Non-linear:** Teams don't progress through Agile in a straight, step-by-step fashion. Progress can loop back, skip steps, or take different directions.
- **Context-dependent:** Each team's environment, goals, and challenges are unique, so adoption looks different for everyone.
- **Teams start at different points:** Some teams may already be familiar with certain Agile practices, while others are starting from scratch.
- **Teams evolve over time:** Teams improve and change as they gain experience, adapt to challenges, and adjust practices.



Major Lines on the Agile Subway Map

- **Extreme Programming (XP):** practices focused on software engineering excellence (e.g., TDD, pair programming, continuous integration).
- **Scrum:** practices related to iterative delivery and team coordination like daily meetings, retrospectives, and backlog refinement.
- **Lean:** practices emphasizing waste minimization, flow, and value delivery.
- **Product Management:** practices around understanding customer needs and shaping product direction (e.g., user stories, personas).
- **DevOps:** practices bridging development and operations for continuous delivery and deployment.

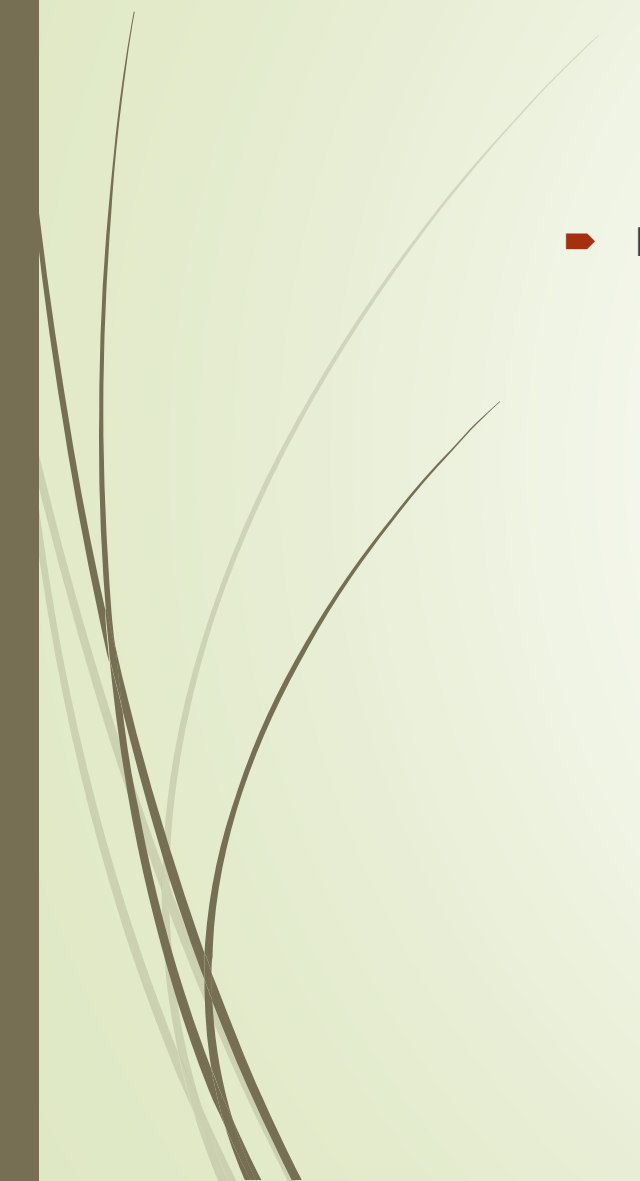


Major Lines on the Agile Subway Map

- **Design:** practices related to understanding users and designing solutions (e.g., story mapping, quick design sessions).
- **Testing** quality and validation practices such as exploratory testing, acceptance tests, and BDD.
- **Teams:** practices that support teamwork and collaboration (e.g., team room, three questions).
- **Fundamentals:** core Agile concepts such as iterative development, incremental delivery, and sustainable pace.



Fundamentals



- Practices that form the core, general Agile mindset and structure.
 - **Team (Agile):** A small, cross-functional group where members collaboratively design, build, and test software. Agile teams are typically self-organizing and share responsibility for outcomes.
 - **Iterative Development:** A timeboxed cycle (often weeks) in which teams plan, build, test, and deliver a piece of working software, then reflect and adapt before the next cycle.
 - **Incremental Development:** A way of building software in small, usable slices, where each increment adds functionality to what was built before.
 - **Version Control:** A system that tracks and manages changes to source code over time, enabling collaboration, branching, merging, and historical traceability.

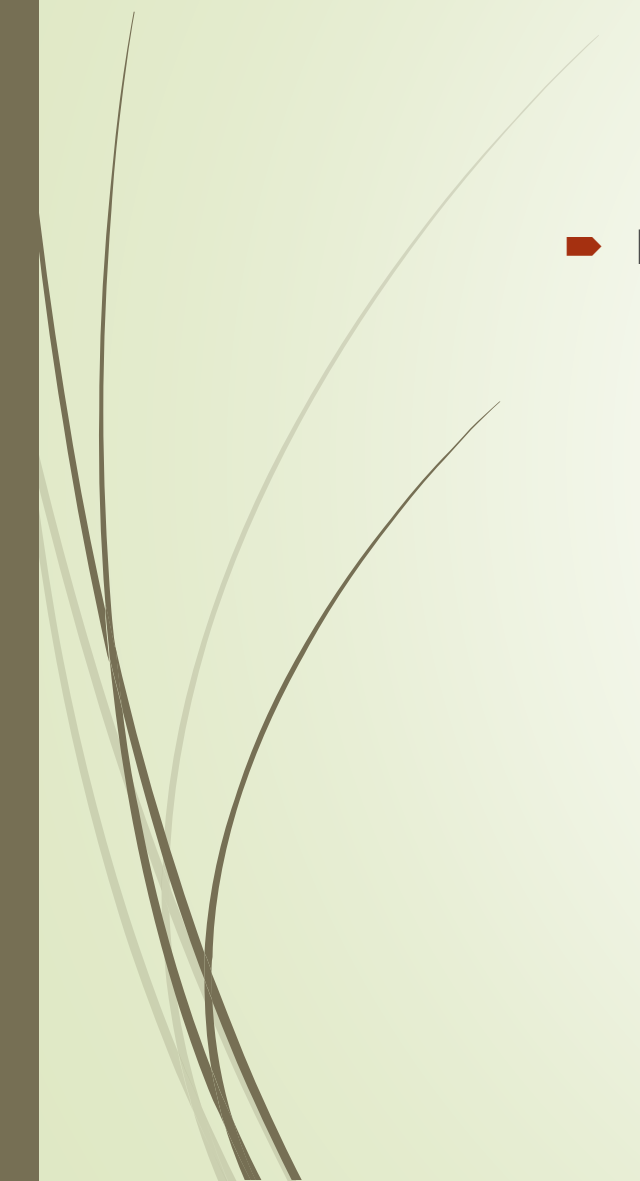


Extreme Programming (XP)

- Technical practices focused on engineering excellence.
 - **Pair Programming:** Two developers work together on one task.
 - **Test-Driven Development (TDD):** Write tests before code.
 - **Continuous Integration:** Frequently merging code into a shared repo.
 - **Simple Design / Rules of Simplicity:** Avoid over-engineering; build simplest thing that works.
 - **Refactoring:** Improving existing code without changing behavior.
 - **Collective Ownership:** Everyone can improve any part of the code.



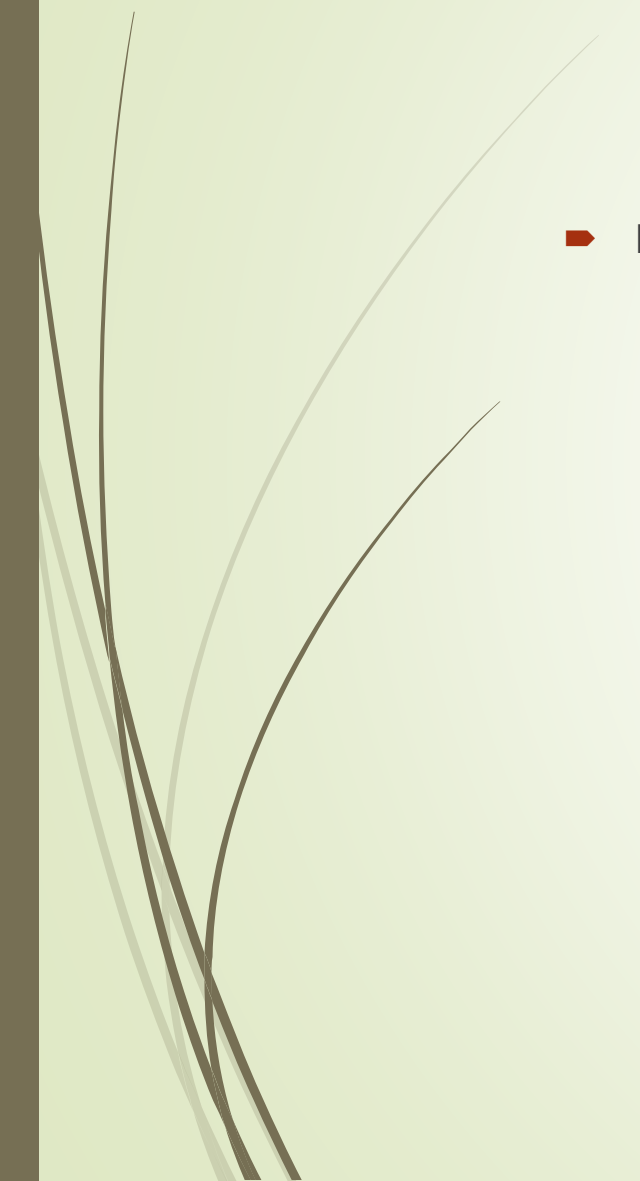
Lean



- Focuses on optimizing flow and eliminating waste.
- **Kanban Board:**
 - A visual workflow board with columns representing stages of work. It helps teams visualize work, limit work in progress (WIP), and manage flow.
- **Lead Time:**
 - Time from start to delivery of work item.
 - The total time from when a customer request is made until it is delivered.



Scrum



- Practices associated with Scrum framework and team coordination.
 - **Backlog** Ordered list of work to be done.
 - **Backlog Refinement** – Ensuring backlog items are clear and ready.
 - **Task Board** – Visual board showing task progress (e.g., To Do, Doing, Done).
 - **Daily Meeting (Daily Standup)** – Short daily team sync.
 - **Burndown Chart** – Visual progress tracker of remaining work.
 - **Three Questions** – Classic standup questions (Yesterday, Today, Blockers).
 - **Scrum of Scrums** – Scaled coordination across multiple teams.
 - **Sign Up for Tasks** – Team members select tasks they will work on.

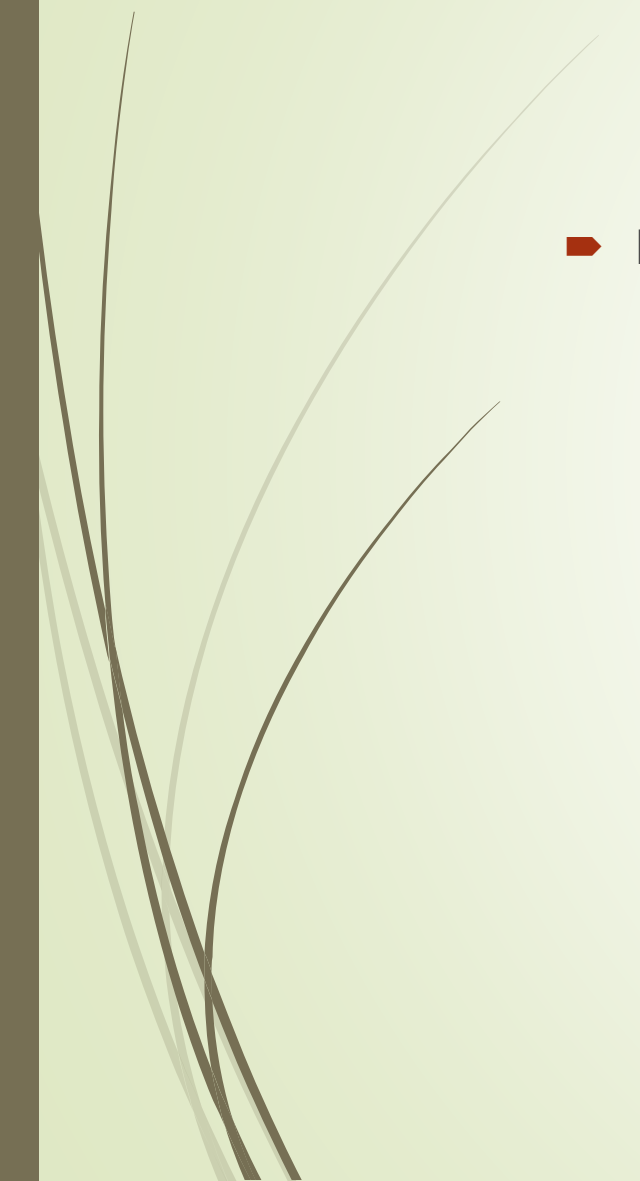


Product Management

- Practices for understanding customer needs and managing what to build.
 - **Backlog Refinement:** An ongoing collaborative activity where the team reviews, clarifies, and prioritizes items in the product backlog so they are well understood and “ready” before entering planning or implementation.
 - **Personas:** Fictitious user profiles that represent key user types. Personas help teams understand user needs and design features with empathy.
 - **Story Mapping:** A visual technique for ordering user stories along axes of workflow sequence and implementation sophistication to reveal scope, priorities, and narrative flow.
 - **Story Splitting:** Breaking large user stories into smaller, more manageable ones that can be completed within an iteration.
 - **User Stories:** Short, simple descriptions of functionality from the user's perspective that guide development and clarify value.
 - **Three C's:** Card – the story title, Conversation – discussion among stakeholders, Confirmation – acceptance criteria/tests
 - **INVEST:** An acronym describing good user stories: Independent, Negotiable, Valuable, Estimable, Small, Testable. These criteria help ensure work items are workable and valuable.



DevOps



- Practices that support rapid delivery and operational reliability.
 - **Automated Build:** Automated compiling and packaging of software.
 - **Continuous Deployment:** Automatically deploying changes to production.
 - **Version Control:** Managing source code versions and history



Testing



- Practices ensuring quality and validating functionality.
 - **Acceptance Testing:** Formal scenarios that describe expected system behavior from the user's perspective. They are often written as examples and can be automated, acting as functional specifications to validate the system's behavior.
 - **Test-Driven Development (TDD):** A programming practice where developers write a failing unit test before writing code, then write only enough code to pass the test, and then refactor.
 - **Acceptance Test-Driven Development (ATDD):** A collaborative practice where the customer, developers, and testers define the acceptance tests before implementation. These acceptance tests become requirements that guide development and verify that the system behaves as intended.
 - **Behavior-Driven Development (BDD):** An evolution of TDD/ATDD that focuses on defining and verifying software behavior in business terms. It emphasizes collaboration and uses examples to specify requirements in a way that all team members understand.

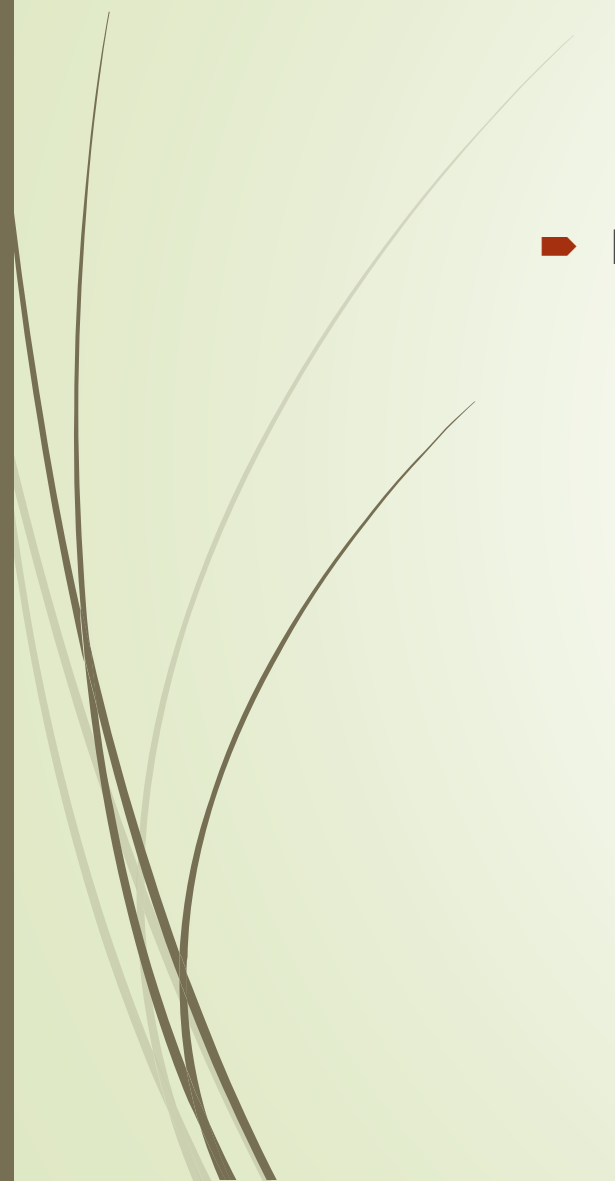


Testing

- ▶ Practices ensuring quality and validating functionality.
 - ▶ **Given-When-Then:** Syntax for defining BDD scenarios. A structured template for writing acceptance tests or behavior examples using: Given (context), When (action), Then (expected result). This format helps clarify requirements and scenarios.
 - ▶ Given my bank account is in credit, and I made no withdrawals recently, When I attempt to withdraw an amount less than my card's limit, Then the withdrawal should be complete without errors or warnings
 - ▶ **Exploratory Testing:** Learning through simultaneous test design and execution.
 - ▶ **Usability Testing:** Testing with real or representative users to evaluate how easy and effective a product is in achieving user goals.
 - ▶ **Unit Tests:** Automated tests written to verify a small, isolated piece of functionality (a “unit”) behaves correctly every time changes are made.
 - ▶ **Mock Objects:** Lightweight simulated objects used in testing to replicate the behavior of real dependencies (e.g., services, databases) so tests run quickly and focus on the unit under test.



Design



- Practices to clarify requirements and architecture before/during building.
 - **Quick Design Session:** A brief design discussion among team members to make essential design decisions early, often used to reduce rework and clarify structure.
 - **Simple Design:** A principle that encourages designing only for current requirements and not adding unnecessary complexity.
 - **Ubiquitous Language:** A shared language developed by the team and stakeholders (often from domain-driven design) to avoid ambiguity and improve communication throughout the project.



Design



- Practices to clarify requirements and architecture before/during building.
 - **Rules of Simplicity** are Kent Beck's criteria for judging good design: code should pass all automated tests, avoid duplication, clearly express responsibilities, and use the smallest design that satisfies these conditions.
 - **CRC Cards** are a collaborative object-oriented design technique where team members use index cards to represent classes, their responsibilities, and collaborators, and then role-play interactions to quickly explore and validate design ideas.



Design - Simple Example: Online Login Feature

- CRC Cards
 - UserInterface
 - **Responsibility:** Collect user input
 - **Collaborator:** AuthenticationService
 - AuthenticationService
 - **Responsibility:** Validate credentials
 - **Collaborator:** UserRepository
 - UserRepository
 - **Responsibility:** Retrieve user data

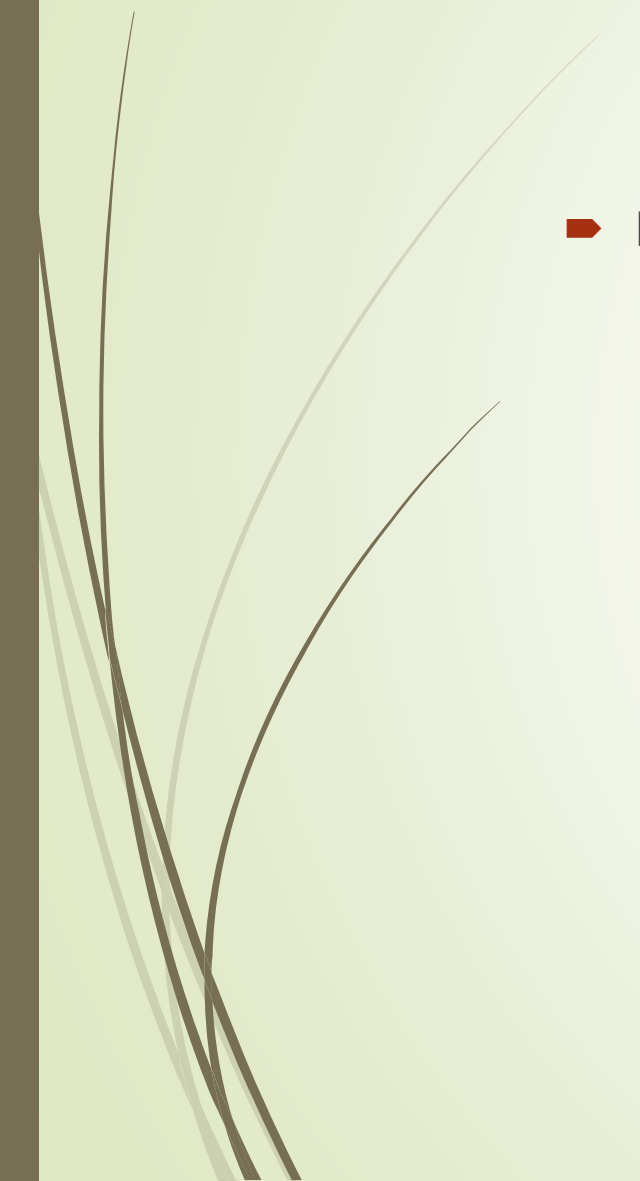


Design - Simple Example: Online Login Feature

- ▶ Team members each take a card and speak as that class:
 - ▶ **UserInterface:**
 - ▶ “I received a username and password from the user. AuthenticationService, can you verify these credentials?”
 - ▶ **AuthenticationService:**
 - ▶ “Sure. UserRepository, please find the user record for this username.”
 - ▶ **UserRepository:**
 - ▶ “Here is the user data. The password hash matches.”
 - ▶ **AuthenticationService:**
 - ▶ “Credentials are valid. UserInterface, you may log the user in.”



Teams



- Practices that support teamwork, collaboration, and team processes.
 - **Team:** A small, cross-functional group where members collaboratively design, build, and test software. Agile teams are typically self-organizing and share responsibility for outcomes.
 - **Team Room:** A shared physical or virtual workspace where the team collocates for communication, visibility of information, and real-time collaboration.
 - **Facilitation:**
 - Creating the conditions for effective group processes, in the pursuit of the objectives for which the meeting was convened.
 - Guiding group conversations and decision making.



References

- ▀ Agile Alliance: <https://agilealliance.org/>